

1. RSA Encryption Analysis

Given:

- $p = 11$
- $q = 13$
- $e = 7$
- Message $M = 9$

(a) Compute n and $\phi(n)$

$$n = p \times q = 11 \times 13 = 143$$

Euler Totient:

$$\begin{aligned}\phi(n) &= (p - 1)(q - 1) \\ &= (11 - 1)(13 - 1) \\ &= 10 \times 12 = 120\end{aligned}$$

Answer:

- $n = 143$
- $\phi(n) = 120$

(b) Find the private key d

Given

$$e = 7$$

Find d such that

$$7d \equiv 1 \pmod{120}$$

Try multiples:

$$\begin{aligned}7 \times 103 &= 721 \\ 721 &= 120 \times 6 + 1\end{aligned}$$

Hence,

$$d = 103$$

Private Key = (103,143)

(c) Encrypt $M = 9$

Formula:

$$C = M^e n$$
$$C = 9^7 143$$

Using modular arithmetic,

$$9^7 = 4782969$$
$$4782969 143 = 48$$

Ciphertext

$$\boxed{C = 48}$$

(d) Decrypt

Formula

$$M = C^d n$$
$$M = 48^{103} 143$$

Using modular exponentiation,

$$M = 9$$

Recovered message equals original.

Answer

Original Message = **9**

Recovered Message = **9**

✓ Verified.

(e) Effect of changing e

- Small e (3 or 5)
 - Faster encryption
 - Can be vulnerable if padding is not used.
- Moderate $e = 65537$
 - Standard choice.
 - Fast and secure.

- Large e
 - Slower encryption.
 - Slightly better resistance to some attacks but usually unnecessary.

Conclusion:

Use **65537** because it provides the best balance of speed and security.

2. Block Cipher Mode Evaluation

Given

Plaintext blocks

$$P = [1010, 1010, 1111\ 1010]$$

Key

$$K = 1100$$

IV

$$0000$$

Assume encryption is **XOR with key**.

(a) ECB Mode

Formula

$$C = P \oplus K$$

Block 1

1010

$$\oplus 1100$$

=0110

Block 2

1010

$$\oplus 1100$$

=0110

Block 3

1111

$$\oplus 1100$$

=0011

Block 4

1010

$\oplus 1100$

=0110

ECB Ciphertext

[0110, 0110, 0011 0110]

CBC Mode

Formula

$$C_i = (P_i \oplus C_{i-1}) \oplus K$$

IV = 0000

Block 1

$1010 \oplus 0000 = 1010$

$1010 \oplus 1100 = 0110$

C1=0110

Block 2

$1010 \oplus 0110 = 1100$

$1100 \oplus 1100 = 0000$

C2=0000

Block 3

$1111 \oplus 0000 = 1111$

$1111 \oplus 1100 = 0011$

C3=0011

Block 4

$1010 \oplus 0011 = 1001$

$1001 \oplus 1100 = 0101$

C4=0101

CBC Ciphertext

[0110, 0000, 0011 0101]

(b) Comparison

ECB

0110 0110 0011 0110

CBC

0110 0000 0011 0101

Repeated plaintext blocks produce identical ciphertext only in ECB.

(c) Which mode is better?

CBC hides patterns better because each plaintext block is XORed with the previous ciphertext block before encryption. Even identical plaintext blocks produce different ciphertexts. ECB encrypts identical plaintext blocks into identical ciphertext blocks, revealing patterns. Therefore, CBC provides better security than ECB.

3. DES vs 3-DES vs Blowfish

Given

Algorithm Time/block

DES 2 ms

3-DES 6 ms

Blowfish 1 ms

1000 blocks

(i) Total Encryption Time

DES

$$1000 \times 2 = 2000 \text{ ms}$$

= 2 seconds

3-DES

$$1000 \times 6 = 6000 \text{ ms}$$

= 6 seconds

Blowfish

$$1000 \times 1 = 1000 \text{ ms}$$

= 1 second

Final Table

Algorithm	Key Size	Time/block	1000 Blocks
DES	56 bits	2 ms	2000 ms (2 s)
3-DES	168 bits	6 ms	6000 ms (6 s)
Blowfish	128 bits	1 ms	1000 ms (1 s)

(ii) Trade-offs

DES

- Weak security
- Fast
- Easily broken today

3-DES

- Stronger than DES
- Slowest
- High computational cost

Blowfish

- Strong security
- Fastest
- Efficient for software implementation

(iii) Recommendation

Blowfish is recommended because it provides strong security with the fastest encryption speed, making it suitable for secure real-time communication.

4. Diffie–Hellman Key Exchange

Given

- $p = 23$
- $g = 5$
- A private = 6
- B private = 15

(d) Public Keys

User A

$$\begin{aligned}A &= g^6 \text{ } 23 \\5^6 &= 15625 \\15625 \text{ } 23 &= 8\end{aligned}$$

Public Key A

$$\boxed{8}$$

User B

$$B = 5^{15} \text{ } 23$$

Using modular exponentiation,

$$5^{15} \text{ } 23 = 19$$

Public Key B

$$\boxed{19}$$

(e) Shared Secret

A computes

$$19^6 \text{ } 23 = 2$$

B computes

$$8^{15} \text{ } 23 = 2$$

Shared Secret

2

(f) Man-in-the-Middle Attack

1. Eve intercepts A's public key (8).
2. Eve sends her own public key to B.
3. Eve intercepts B's public key (19).
4. Eve sends another fake public key to A.
5. A and B each establish separate shared secrets with Eve instead of with each other.
6. Eve decrypts, reads, modifies, and re-encrypts messages between them without their knowledge.

(g) Prevention

Use **authenticated Diffie–Hellman**, such as:

- Digital Signatures
- Public Key Certificates
- TLS/SSL authentication

These methods verify identities and prevent Man-in-the-Middle attacks.

5. ECC vs RSA Comparison

Given

- RSA = 1024 bits
- ECC = 160 bits
- IoT device with limited processing

(d) Computational Cost

RSA

- Large keys
- More CPU operations
- Slower key generation and decryption

ECC

- Much smaller keys
- Faster computations
- Lower CPU usage

ECC has lower computational cost.

(e) Power and Memory Consumption

RSA

- Higher memory requirement
- Higher battery consumption
- Larger storage for keys

ECC

- Small keys
- Less RAM usage
- Lower power consumption
- Faster execution

ECC consumes less power and memory than RSA.

(f) Best Cryptosystem for IoT

ECC is better for IoT devices because it provides security equivalent to RSA with much smaller key sizes. This reduces computation time, memory usage, power consumption, and communication overhead, making it ideal for resource-constrained devices.

Final Answers (Summary)

1. RSA

- a) $n = 143$, $\phi(n) = 120$
- b) $d = 103$
- c) Ciphertext = **48**
- d) Decrypted Message = **9**
- e) Larger or standard e (commonly **65537**) improves practical security while balancing encryption speed.

2. Block Cipher

- ECB = [0110, 0110, 0011, 0110]
- CBC = [0110, 0000, 0011, 0101]

- CBC hides patterns better.

3. Performance

- DES = **2 s**
- 3-DES = **6 s**
- Blowfish = **1 s**
- Best choice: **Blowfish**

4. Diffie–Hellman

- Public Key A = **8**
- Public Key B = **19**
- Shared Secret = **2**
- Prevent MITM using **Digital Signatures/Certificates (Authenticated Diffie–Hellman or TLS)**.

5. ECC vs RSA

- ECC has lower computational cost.
- ECC uses less power and memory.
- Best for IoT: **ECC**.